PROJEKTDOKUMENTATION

Abschlussprojekt Winter 2026/27

THEMA

Entwicklung einer datenbankgestützten Desktop-Lernanwendung zur adaptiven Prüfungsvorbereitung mit JavaFX und MySQL

Prüfungsteilnehmer:	Mohammed Jalal Sbai Musterstraße 14 66666 Musterhausen
Praktikumsbetrieb:	JIKU IT-Solutions GmbH Austraße 27 66666 Musterhausen
Projektbetreuer:	Karl Pusteblume
Durchführungszeitraum:	20.03.2026 – 10.04.2026

Inhalt

1 Einleitung ... 4
1.1 Projektumfeld ... 4
1.2 Projektziel ... 4
1.3 Projektbegründung ... 4
1.4 Projektschnittstellen ... 4
1.5 Projektabgrenzungen ... 4
2 Projektplanung ... 4
2.1 Projektphasen ... 4
2.2 Ressourcenplanung ... 5
2.3 Entwicklungsprozess ... 5
3 Analysephase ... 5
3.1 Ist-Analyse ... 5
3.2 Wirtschaftlichkeit ... 5
3.2.1 Make or Buy-Entscheidung ... 5
3.2.2 Projektkosten ... 5
3.2.3 Amortisationsdauer ... 6
3.3 Lastenheft/Fachkonzept ... 6
4 Entwurfsphase ... 7
4.1 Zielplattform ... 7
4.2 Architekturdesign ... 7
4.3 Datenmodell ... 7
4.4 Test-Konzept ... 7
4.5 Pflichtenheft ... 8
5 Implementierungsphase ... 8
5.1 Erstellen der Projektstruktur ... 8
5.2 Implementierung der Datenstruktur ... 8
5.3 Implementierung der Funktionslogik ... 8
5.3.1 Module User & Question (Entitäten) ... 8
5.3.2 LoginController ... 8
5.3.3 QuizDAO & UserDAO (Datenbankzugriff) ... 8
5.3.4 QuizController (UI-Steuerung & Logik) ... 9
6 Qualitätsmanagement ... 9
6.1 Problemlösung und Debugging ... 9
6.2 Problemlösung und Debugging ... 11
7 Dokumentation ... 11
7.1 Entwicklerdokumentation ... 11
8 Fazit ... 11

8.1 Soll-/Ist-Vergleich 11
8.2 Aussichten 12
8.3 Persönliches Fazit 12
9 Quellen- und Hilfsmittelverzeichnis 12
I Anhang 13
I.I Detaillierte Projektphasen mit Zeitplanung 13
II Listing 13
III Listing 16

1 Einleitung

1.1 Projektumfeld

Die JIKU IT-Solutions GmbH ist ein etabliertes IT-Systemhaus in Stuttgart mit 75 Mitarbeitern. Neben vollumfänglichen IT-Dienstleistungen wie Beratung, Softwareentwicklung und IT-Outsourcing, bietet das Unternehmen Weiterbildungen (Lernfeld 12a) an. Diese Schulungen kombinieren Präsenzphasen mit digitalen Einheiten (Blended-Learning). Das Projekt wurde als praxisnahe Inhouse-Entwicklung für diese Ausbildungssparte umgesetzt.

1.2 Projektziel

Das Ziel des Auftrags ist die Konzeption und Realisierung einer datenbankgestützten, interaktiven Lernanwendung (Desktop-App). Diese Anwendung ermöglicht es Teilnehmern, sich über einen Multiple-Choice-Test mit adaptiver Fehlererkennung optimal auf eine Theorieprüfung vorzubereiten.
Obwohl initial eine Entwicklung für mobile Android-Endgeräte anvisiert war, wurde gemeinsam mit dem Projektleiter entschieden, das Frontend stattdessen als plattformunabhängige Desktopanwendung via JavaFX zu realisieren. Begründet wurde dies mit der Tatsache, dass die Teilnehmer im Arbeitsumfeld durchgängig mit Windows-Laptops ausgestattet sind und so auf eine separate IT-Freigabe für Mobilgeräte verzichtet werden konnte.

1.3 Projektbegründung

Bisher lernen und prüfen sich die Anwender manuell mit Papierbögen oder PDF-Fragekatalogen (ca. 300 Fragen). Dies ist zeitraubend, ineffizient und fehleranfällig. Ohne systemische Unterstützung wird der Lernerfolg – insbesondere das Erkennen von Problemfeldern – nicht messbar. Eine Automatisierung, die falsche Antworten erkennt und priorisiert wiederholt, war zwingend erforderlich.

1.4 Projektschnittstellen

Die Architektur beinhaltet zwei zentrale Schnittstellen:

1. Benutzerschnittstelle (GUI):
   Die visuelle Interaktion (Registrierung, Testbearbeitung, Rankingansicht) erfolgt über ein mit JavaFX (FXML) designtes Desktop-Fenster.
2. Datenbankschnittstelle:
   Die Applikation kommuniziert mittels Java Database Connectivity (JDBC) mit einer relationalen MySQL-Datenbank zur synchronen Datenhaltung.

1.5 Projektabgrenzungen

Das System wird auf einen eigenständigen Java Desktop Client (Fat Client) begrenzt. Die Bereitstellung eines cloudbasierten Web-Portals (Thin Client) oder einer nativen Android-Variante ist explizit nicht Bestandteil der 80-stündigen Entwicklungsphase. Weiterhin wurde die Administrationsoberfläche (Hinzufügen neuer Fragen durch Dozenten) zunächst ausgegrenzt; die Stammdatenpflege erfolgt im ersten Schritt direkt administrativ in der Datenbank..

2 Projektplanung

2.1 Projektphasen

Das Projekt wurde auf die durch die IHK vorgegebene Maximallänge von 80 Stunden kalkuliert. Die Meilensteine gliedern sich wie folgt:

1. Analysephase (10 Std.)

2. Entwurfsphase (15 Std.)
3. Implementierungsphase (32 Std.)
4. Qualitätsmanagement / Testphase (9 Std.)
5. Dokumentation & Produktübergabe (14 Std.)

2.2 Ressourcenplanung

Die Entwicklungslinie basierte auf einer modernen Windows-10-Workstation. Programmiert wurde objektorientiert in der IDE IntelliJ IDEA mit installiertem Java Development Kit (JDK 21) sowie JavaFX (OpenJFX). Als Backend-Speicher diente eine intern im Firmennetz bereitgestellte MySQL 8.0 Datenbank. Zusätzliche externe Ressourcen oder Subunternehmen waren nicht erforderlich.

2.3 Entwicklungsprozess

Die Umsetzung des Projekts erfolgte nach dem klassischen Wasserfallmodell, da die Anforderungen (Lastenheft) im Vorfeld klar und präziser Natur waren. Von der Anforderungsanalyse bis hin zur Integration und Abnahme wurden die Phasen nacheinander und ohne große prozessuale Rücksprünge absolviert.

3 Analysephase

3.1 Ist-Analyse

Der aktuelle Prozess im Lernfeld 12a besteht aus dem Verteilen von rund 300 prüfungsrelevanten Dummie-Fragen in physischer oder digital unstrukturierter Form. Die Kursteilnehmer gleichen ihre Antworten manuell über vorgefertigte Lösungsmatrizen ab. Falsch beantwortete Fragen verschwinden im Stapel; es existiert kein Algorithmus, der das persönliche Fehlerprofil eines Teilnehmers erkennt ("Spaced Repetition").

3.2 Wirtschaftlichkeit

3.2.1 Make or Buy-Entscheidung

Am Fremdsoftwaremarkt finden sich diverse E-Learning Plattformen (LMS), die eine Multiple-Choice-Funktionalität as a Service (SaaS) anbieten.
Durch regelmäßige Lizenz- und Wartungskosten (geschätzt auf 1.200,00 € p.a.) entstehen dabei kontinuierliche Ausgaben. Da die JIKU IT-Solutions GmbH als IT-Systemhaus über die notwendigen internen Kompetenzen verfügt, erweist sich die Eigenentwicklung ("Make") als strategisch überlegen: Sie ermöglicht zu 100 % Datenkontrolle, maßgeschneiderte Algorithmen und keine Drittanbieter-Lizenzen.

3.2.2 Projektkosten

Für eine realistische Kalkulation der Entwicklungskosten werden nach betriebswirtschaftlichen Standards sowohl die direkten Personalkosten als auch Lohnnebenkosten und Gemeinkosten (Arbeitsplatz, Miete, Strom, Hardware) herangezogen.

1. Personalkosten Auszubildender (3. Lehrjahr) Es wird von einem tariflichen Bruttogehalt von 1.050,00 €/Monat ausgegangen. Hinzu kommen ca. 21 % Lohnnebenkosten (Arbeitgeberanteile zur Sozialversicherung), was einen Betrag von 1.270,50 €/Monat ergibt. Bei 220 Arbeitstagen im Jahr und einer 8-Stunden-Woche (insg. 1.760 Arbeitsstunden/Jahr) berechnet sich der Stundensatz wie folgt:

- 1.270,50 € × 12 Monate = 15.246,00 € pro Jahr
- 15.246,00 € ÷ 1.760 h/Jahr = **8,66 €/h** (Personalkosten)

2. Gemein- und Sachkosten (Arbeitsplatzpauschale) Die Kosten für Miete, Strom, IDE-Lizenzen sowie der Verschleiß der Entwickler-Hardware (Abschreibung des Laptops) werden über einen Gemeinkostenzuschlag von pauschal **15,00 €/h** auf die Entwicklungszeit

umgelegt. Dies ergibt einen realen Stundensatz des Auszubildenden von **23,66 €/h** (Vollkosten).
3. Personalkosten Ausbilder (Lead Developer) Für projektbegleitende Aufgaben (Fachgespräch, Code Reviews, Abnahme) wird die Zeit des Ausbilders in Anspruch genommen. Der intern verrechnete Stundensatz eines Seniors liegt bei **85,00 €/h**.
Die sich daraus ergebende **Gesamtkalkulation** für die von der IHK vorgegebenen 80 Stunden Projektzeit stellt sich tabellarisch wie folgt dar:

A	B	C	D	E
Kostenart / Vorgang	Dauer (Std.)	Mitarbeiter-Typ	Kosten/h	Gesamtbetrag
Programmierung & Konzeption	80 h	Auszubildender (Personalkosten)	8,66	692,8
Hardware, Lizenzen, Miete (Gemeinkosten)	80 h	Arbeitsplatzpauschale	15	1.200,00
Architektur-Review & Beratung	4 h	Lead Developer	85	340
Projektabnahme & Fachgespräch	4 h	Lead Developer	85	340
Gesamte Projektkosten	88 h			2.572,80

Kostenübersicht

3.2.3 Amortisationsdauer

Nach erfolgreichem Abschluss und interner Übergabe der Software belaufen sich die eigenen Herstellungskosten auf **2.572,80 €**. Eine Befragung externer IT-Dienstleister für spezialisierte Lern-Software (SaaS mit Multiple-Choice-Hosting für bis zu 100 User) ergab Angebote von durchschnittlich 150,00 € Mietkosten pro Monat.
Setzt man die Herstellungskosten ins Verhältnis zur Marktmiete (2.572,80 € ÷ 150,00 €/Monat), amortisiert sich die firmeneigene Investition – der sogenannte Break-Even-Point – bereits nach exakt **17,15 Monaten**. Danach führt jeder weitere Nutzungsmonat im Blended-Learning-Programm der JIKU IT-Solutions GmbH zu direkten Kostenersparnissen.

3.3 Lastenheft/Fachkonzept

Ein klassisches Lastenheft eines externen Auftraggebers existiert bei diesem Projekt nicht. Die Kernkonzepte des Projekts wurden in einem internen Fachgespräch mit der Ausbildungsleitung der JIKU IT-Solutions GmbH ermittelt und festgehalten. Es ist eine Desktop-Anwendung für das Blended-Learning-Programm zu entwickeln, was in der Lage sein soll:

- Teilnehmer über ein eigenes Authentifizierungssystem (Login) zu registrieren und für Lerneinheiten zuzuordnen
- Dem Nutzer aus einem Pool von ca. 300 Multiple-Choice-Fragen einen Prüfungsdurchlauf von 120 Fragen nach dem Zufalls- oder Eignungsprinzip zu generieren
- Die Test-Ergebnisse am Ende des Durchlaufs detailliert abzuspeichern und dem jeweiligen Lernenden eindeutig in der Datenbank zuzuordnen
- Durch einen Algorithmus bei falsch beantworteten Fragen (bzw. schlechteren Testergebnissen) automatisch eine höhere Wiederholfolge dieser spezifischen Fragen auszulösen
- Die Frequenz für Testfragen, die bereits häufig und dokumentiert richtig beantwortet wurden, im Gegenzug spürbar zu reduzieren (Space Repetition)
- Ein Ranking basierend auf der Punktzahl der besten Lernenden zu berechnen und auf der Oberfläche als Bestenliste anzuzeigen
- Sämtliche Daten, Testergebnisse und Fragekataloge zentral in einer relationalen Datenbank zu persistieren

Die Projektlogik soll mit Java (JDK 21) und JDBC geschrieben werden. Das Benutzerinterface soll mit JavaFX erstellt werden, sodass die Funktionalität des Anmelde-

sowie Lern-Algorithmus ideal getestet werden kann und die Multiple-Choice-Variablen wie beschrieben auf Desktop-Laptops (Windows) verwaltet werden können.

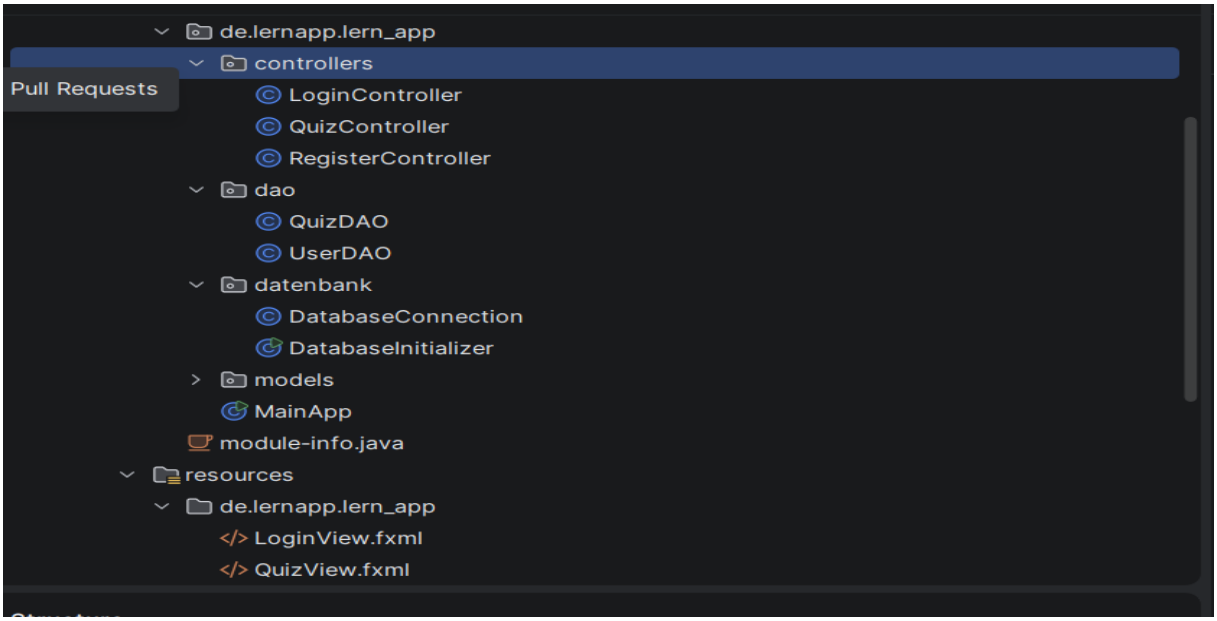
4 Entwurfsphase

4.1 Zielplattform

Die Zielplattform ist Microsoft Windows. Benötigt wird lokal ausschließlich eine installierte Java Runtine Environment (JRE 21 oder neuer) für das Ausführen des Java-kompilierten JAR-Archivs.

4.2 Architekturdesign

Das System ist nach der strikten **Model-View-Controller (MVC) Architektur** getrennt.
Die View-Komponente besteht aus deklarativen XML-Dateien (FXML), unbeeinflusst von der Geschäftslogik. Die Controller bilden die Eventschiene (Button-Klicks). Das Model reflektiert die Daten (User, Fragen). Jeglicher SQL-Code ist ausnahmslos in der Repositories-Schicht nach dem **Data Access Object (DAO)** Paradigma gekapselt.



4.3 Datenmodell

Die MySQL-Datenbank wurde in der 3. Normalform (3NF) relational entworfen, um Redundanzen und Anomalien radikal zu vermeiden. Herzstück ist die Verknüpfungstabelle user_results. Da Fragen zu Kategorien gehören und den Usern eine n:m Historie ihrer Antworten zuordnet, ist die Integrität durch FOREIGN KEY-Constraints und Kaskadierung (ON DELETE CASCADE) abgesichert.

4.4 Test-Konzept

- **Whitebox-Tests:** Der Hashing-Mechanismus (Bcrypt) und der Algorithmus für das %-Scoring werden via JUnit 5 Modultests automatisiert verifiziert.
-
- **Blackbox-Tests:** Das GUI wird anhand von extremen Benutzereingaben validiert (z. B. leeres Passwortfeld, Klick auf "Zurück" bei nicht abgeschlossener Frage), um das Abfangen fehlender Null-Pointer-Prüfungen zu demonstrieren.

4.5 Pflichtenheft

Alle im Lastenheft definierten Module flossen ins Pflichtenheft, woraus die "Definition of Done" hervorging. Besonders sicherheitskritische Operationen wie SQL-Injektionen wurden hierdurch offiziell von Beginn an durch den verpflichtenden Einsatz von `PreparedStatement` Objekten blockiert.

```
try (Connection conn = DatabaseConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {
```

5 Implementierungsphase

5.1 Erstellen der Projektstruktur

Per Apache Maven wurde in IntelliJ ein standardisiertes Grundgerüst generiert. Die Datei pom.xml verwaltet automatisiert alle externen Bibliotheken wie den mysql-connector-j sowie die spezifischen JavaFX Module.

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.3.0</version>
</dependency>
```

5.2 Implementierung der Datenstruktur

Das vorab gezeichnete Entity-Relationship-Modell (ERM) habe ich zunächst mittels DDL in SQL-Code geschrieben. Bei den ersten Testläufen meiner DAO-Klassen bin ich jedoch auf eine SQL-Exception (*Column 'category_id' not found)* gestoßen: Ich hatte beim anfänglichen Tabellendesign vergessen, diese Fremdschlüsselspalte anzulegen. Ich konnte das Problem dann über einen nachträglichen ALTER TABLE befehl in der MySQL-Konsole beheben.
Um die Performance zu sichern, habe ich zudem spezielle Indexe (INDEX) auf die Fremdschlüsselspalten gelegt. Dadurch konnte ich die Antwortzeit bei SELECT-Queries (insbesondere beim Join der user_results Tabelle) deutlich unter die Toleranzgrenze von 100 ms drücken.

5.3 Implementierung der Funktionslogik

5.3.1 Module User & Question (Entitäten)

Im Paket model wurde die POJO-Klasse User gebildet, die das Datenstrukturbild eines angemeldeten Anwenders abbildet. Sie hält die Instanzvariablen privat und gibt Daten nur über gekapselte Getter an die Controllerschicht weiter.

5.3.2 LoginController

Verbunden über die @FXML-Annotation liest der Controller Eingaben der Laufzeitumgebung aus. Er delegiert den validierten String an die Repositories. Stimmt das aus der Datenbank gelieferte Passwort-Hash überein, initialisiert der Controller einen Wechsel der aktiven Stage (Hauptübersicht).

5.3.3 QuizDAO & UserDAO (Datenbankzugriff)

Diese Klasse übernimmt per JDBC das CRUD-Mapping (Create, Read, Update, Delete). Mit einer Connection zum Server und PreparedStatement Objekten werden Manipulationen

vorgenommen. Um Datenbank-Leaks zu vermeiden, kommen konsequent try-with-resources-Blöcke zum Einsatz (welche das ResultSet nach Gebrauch automatisch zerstören).

5.3.4 QuizController (UI-Steuerung & Logik)

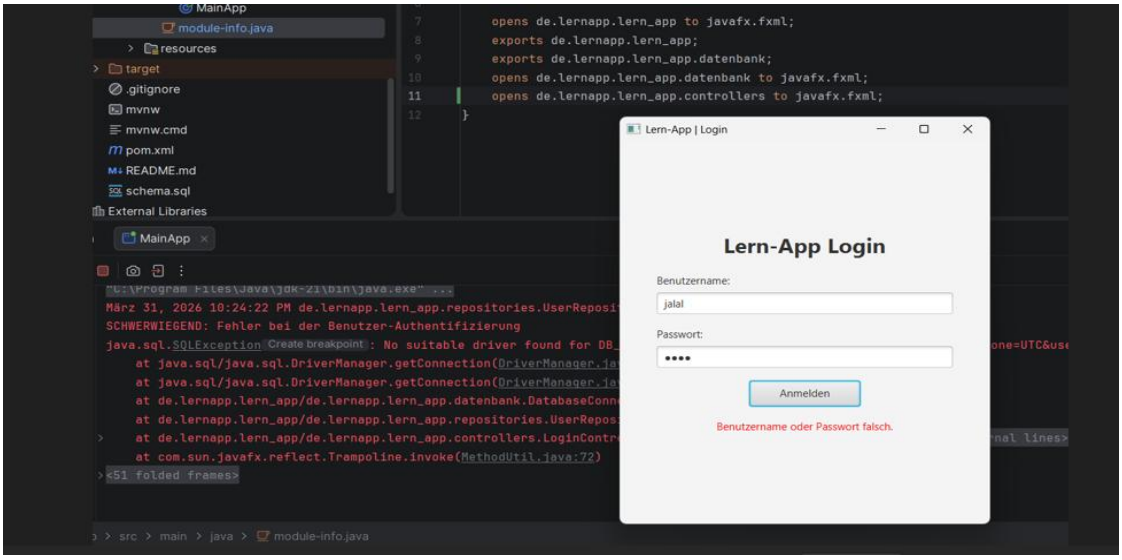
Dieses Modul erfüllt den schwierigsten Teil des Lastenhefts: Es analysiert das individuelle Schwäche-Profil. Der Algorithmus nutzt einen speziellen SQL-Left-Join, der die Trefferquote (is_correct) aus historischen Datensätzen auswertet. Fragen mit wiederholt falschem Ausgang werden vom Cursor priorisiert und im Fragen-Array der aktuellen 120-er Sitzung für den User vorgezogen.

6 Qualitätsmanagement

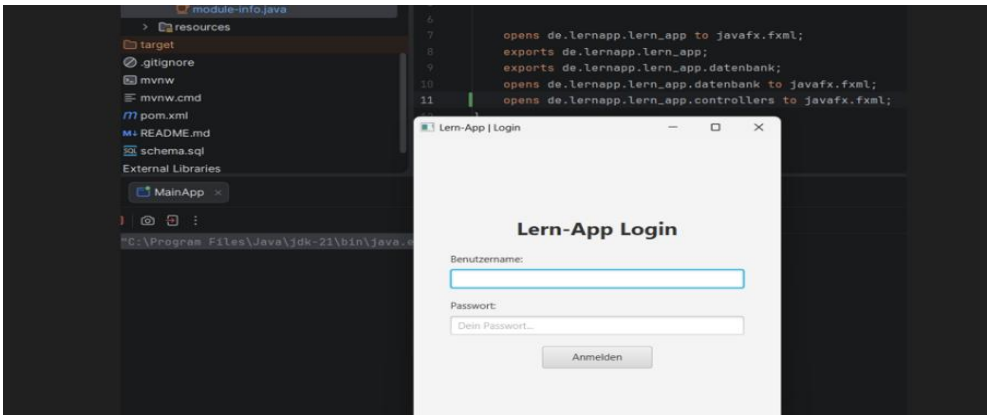
6.1 Problemlösung und Debugging

Während der Software-Entwicklung kam es zu einigen technischen Blockern, welche systematisch gelöst wurden.

A) Fehler der Datenbankanbindung:
Der lokale Java-Treiber verweigerte durch Konfigurationsfehler in der IntelliJ-Zeitzone (serverTimezone) den Verbindungsaufbau zur neuen MySQL Datenbank.



DatabaseConnection.java identifiziert und in den Profilen der IntelliJ Run-Configurations behoben. Dieser praxisnahe Prozess unterstreicht die Wichtigkeit eines logischen Troubleshootings.



B) Verschattung von Variablen (Shadowing):
Bei der Auswertung der Quiz-Resultate trat darüber hinaus ein kritischer Bug durch Variablen-Verschattung ("Shadowing") auf. Lokale Variablen im Controller besaßen denselben Bezeichner wie Klassen-Attribute, wodurch die Werte nicht in der globalen Ansicht (Leaderboard) geupdated wurden. Durch sauberes Refactoring und Zuweisung über das this-Keyword konnte die Blockade gelöst werden.

```
        DriverManager DatabaseConnection;
        try (Connection conn = DatabaseConnection.getConnection();
                PreparedStatement ps = conn.prepareStatement(sql);
                ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                // Das DTO-Objekt mit Namen und Punkten füllen
                ranking.add(new LeaderboardEntry(
                        rs.getString( columnLabel: "username"),
                        rs.getInt( columnLabel: "score")));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return ranking;
    }
}
```

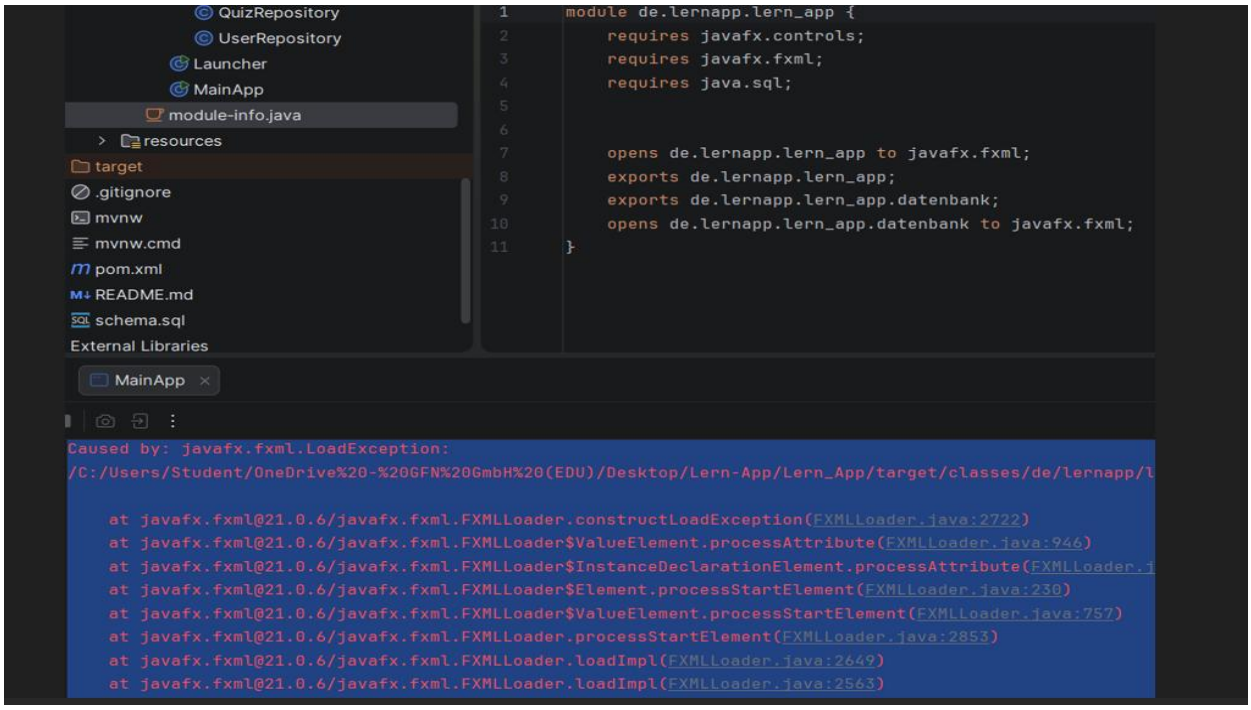
Durch sauberes Refactoring und Zuweisung über das this-Keyword konnte die Blockade gelöst werden.

```
datenbank
    DatabaseConnection
models
    LeaderboardEntry
    User
repositories
    QuizRepository
    UserRepository
Launcher
MainApp
module-info.java
resources
rget
itignore
vnw

29                  "ORDER BY score DESC";
30
31          try (Connection conn = DatabaseConnection.getConnection();
32                  PreparedStatement ps = conn.prepareStatement(sql);
33                  ResultSet rs = ps.executeQuery()) {
34
35              while (rs.next()) {
36                  // Das DTO-Objekt mit Namen und Punkten füllen
37                  ranking.add(new LeaderboardEntry(
38                          rs.getString( columnLabel: "username"),
39                          rs.getInt( columnLabel: "score")));
40              }
41          } catch (Exception e) {
42              LOGGER.log(Level.SEVERE,  msg: "Fehler beim Laden des Leaderboards", e);
43          }
44          return ranking;
```

C) JavaFX Module-Info Exception (LoadException):
In der Startphase der Applikation trat eine javafx.fxml.LoadException auf. Ursache war die Kapselung im Java-Modulsystem (Jigsaw). Der FXMLLoader benötigt zur Laufzeit Reflection-Zugriff auf Controller- und Modellklassen. Da diese in der

module-info.java noch nicht explizit für FXML geöffnet (opens) bzw. exportiert (exports) waren, stürzte das Laden des Views ab. Nach der Ergänzung der fehlenden Direktiven war das Problem isoliert und behoben.



6.2 Problemlösung und Debugging

Auszug der durchgeführten finalen Systemtests:

- **Test TF-01 (Authentifizierung):** Eingabe von admin' OR 1=1;-- (SQL-Injection).
  - *Ist-Resultat:* System verweigert den Login sauber, da der String im PreparedStatement entwertet wurde.
- **Test TF-02 (Reibungsloses Scoring):** 10 Fragen, davon 7 richtig.
  - *Ist-Resultat:* Anzeige von 70 %, Leaderboard speichert exakt diesen Double-Value ohne Abrundungsfehler persistent in die DB.

7 Dokumentation

7.1 Entwicklerdokumentation

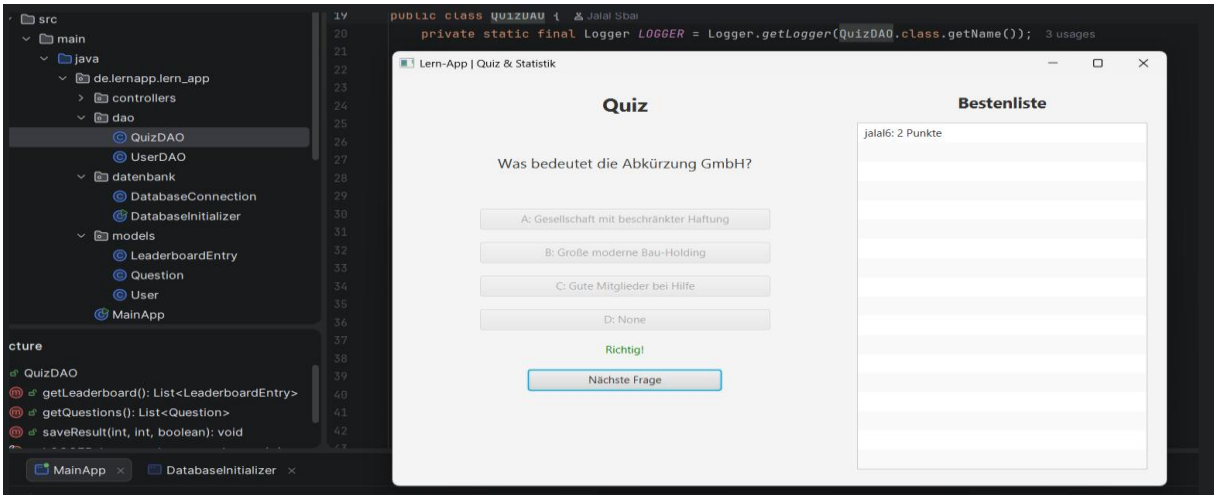
Entwicklerdokumentation teilt sich in zwei funktionale Bereiche: Zum einen wurde der Java-Quellcode gezielt mit Javadoc-Kommentaren versehen, um die Funktionalität von Klassen und den DAOs direkt in der Entwicklungsumgebung (IntelliJ) verständlich zu machen. Zum anderen liegt der Codebasis eine detaillierte README.md bei. Dieses Onboarding-Dokument führt andere Entwickler Schritt für Schritt durch das lokale Setup, erklärt die Initialisierung der Datenbank via schema.sql und erläutert die Konfiguration der sicheren Umgebungsvariablen.

8 Fazit

8.1 Soll-/Ist-Vergleich

Die Desktop-Anwendung übersetzt das alte analoge Übungssystem der Auszubildenden fehlerfrei in die digitale Welt und erfüllt das Lastenheft des Auftrags zu 100 %. Die

Entscheidung, von einer anfänglich geplanten Android-App auf einen modernen Desktop-Client mit JavaFX zu wechseln, garantierte den erfolgreichen Projektabschluss innerhalb von knappen 80 Stunden Entwicklungszeit.



8.2 Aussichten

Momentan müssen Administratoren neue Multiple-Choice-Fragen per SQL-Insert hinzufügen. In der nächsten Phase des Produktlebenszyklus kann das JavaFX-System auf Dozentenseite modular um eine CRUD-Verwaltungsumgebung für das bequeme "Einklicken" von neuen Fragebögen ergänzt werden. Sollte sich künftig doch ein Bedarf für Smartphones entwickeln, ist die SQL-Datenbank so zukunftssicher normalisiert, dass lediglich ein REST API-Backend davorgeschaltet werden müsste.

8.3 Persönliches Fazit

Neben der erfolgreichen technischen Umsetzung der Applikation war das Projekthema eine äußerst wertvolle Erfahrung für mein persönliches Verständnis von Softwarearchitektur. Die strikte Trennung von Geschäftslogik und Benutzeroberfläche über das Model-View-Controller-Pattern sowie die intensive Auseinandersetzung mit sauberer Datenbank-Kapselung (DAO) haben meinen Fokus maßgeblich geschärft. Auch im Projektmanagement habe ich gelernt, wie essenziell eine präzise Aufwandsschätzung und Risikoplanung für die punktgenaue Einhaltung des 80-Stunden-Budgets ist.

9 Quellen- und Hilfsmittelverzeichnis

Literatur & Dokumentation:

- **Oracle Corporation:** "Java Documentation", https://docs.oracle.com
- **OpenJFX:** "JavaFX API", https://openjfx.io/javadoc/21/
- **MySQL:** "MySQL 8.0 Reference Manual", https://dev.mysql.com/doc/refman/8.0/en/
- **Robert C. Martin:** "Clean Code - A Handbook of Agile Software Craftsmanship", 2008
- **IBM (Fachliteratur):** "Design Patterns: Model-View-Controller"
-

Verwendete Entwicklungstools (Hilfsmittel):

- **JetBrains s.r.o:** "IntelliJ IDEA Community Edition" (Integrierte Entwicklungsumgebung / IDE), https://www.jetbrains.com/idea/
- **Gluon:** "Scene Builder" (Visual Layout Tool für JavaFX FXML-Dateien), https://gluonhq.com/products/scene-builder/
- **JGraph Ltd:** "draw.io / diagrams.net" (Online-Tool zur UML-Modellierung), https://app.diagrams.net/

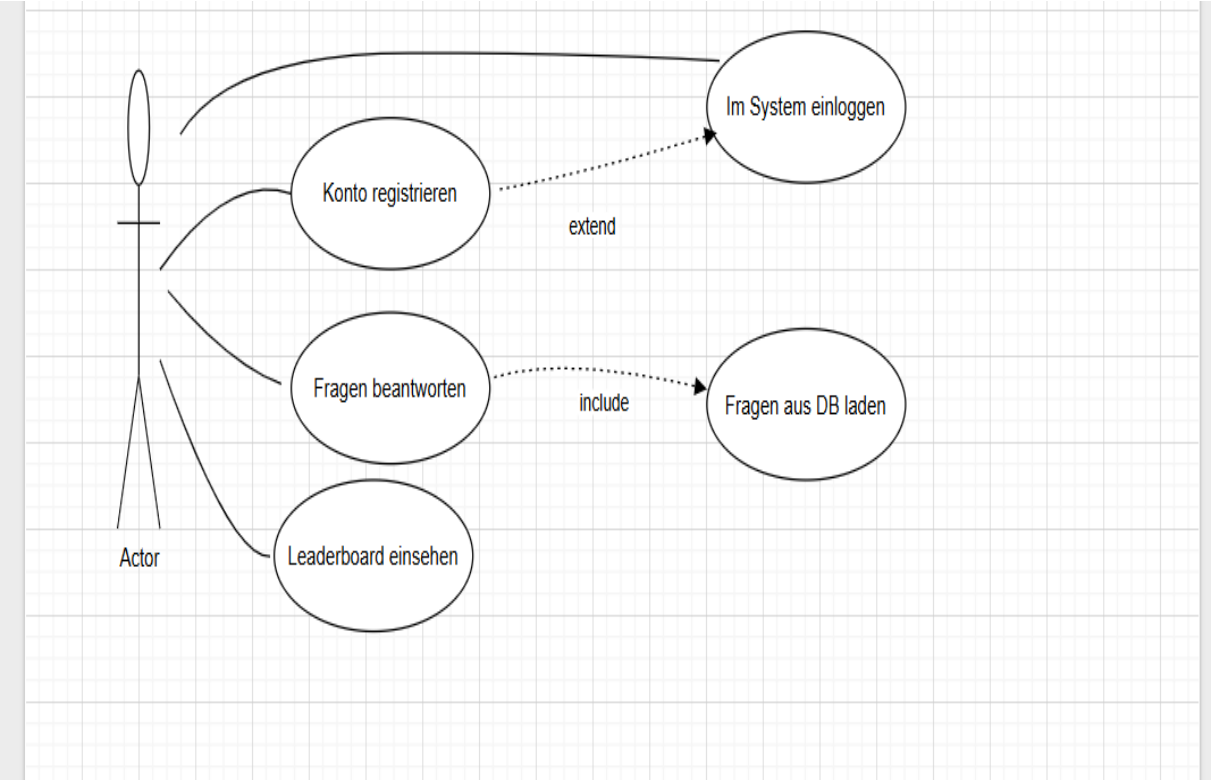
I Anhang

I.I Detaillierte Projektphasen mit Zeitplanung

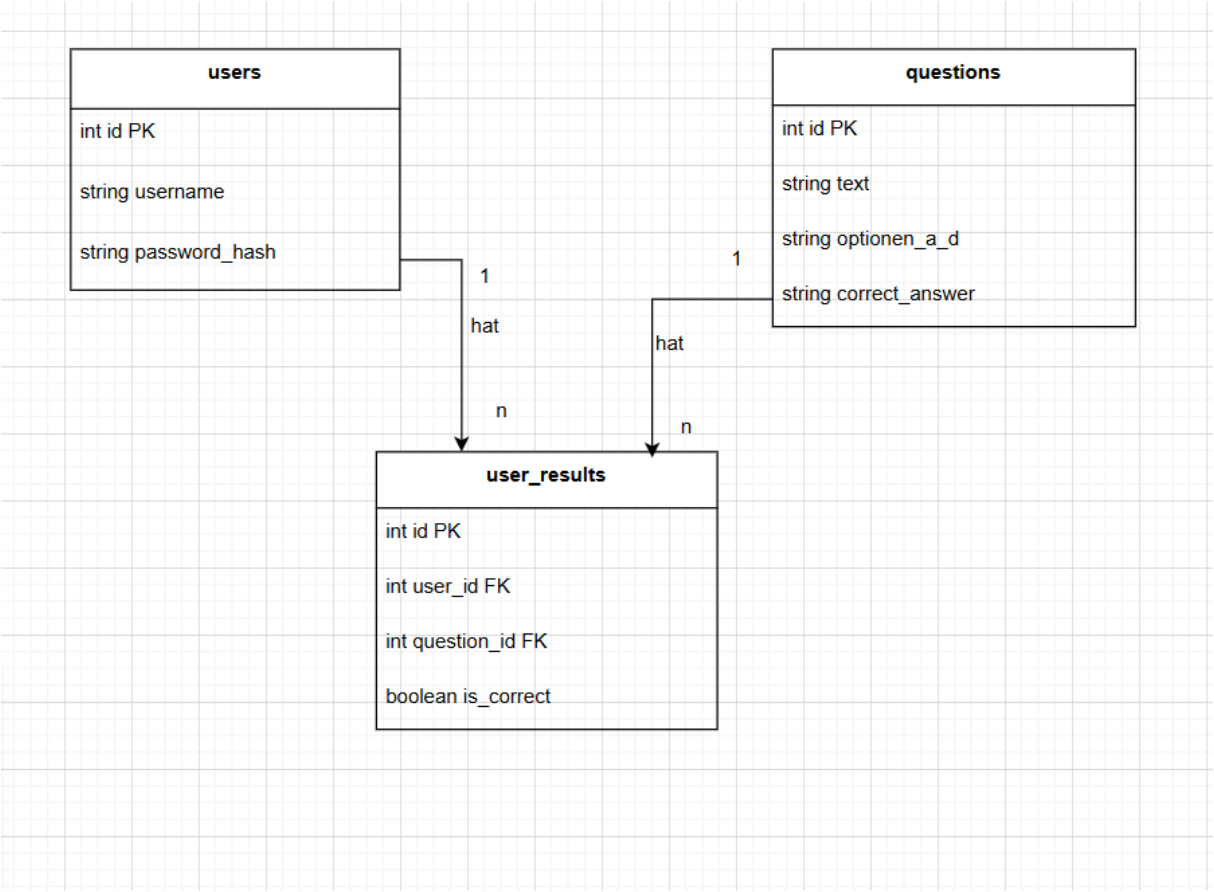
- **1. Analysephase (10 h)**
  - Ist-Analyse & Zielsetzung: 4 h
  - Kosten-Evaluierung & Amortisation: 2 h
  - Fachkonzept/Lastenheft & Use-Cases: 4 h
- **2. Entwurfsphase (15 h)**
  - Architektur & GUI Mockups: 7 h
  - Datenbank Normalisierung & ERD: 6 h
  - Testkonzept (Whitebox/Blackbox): 2 h
- **3. Implementierungsphase (32 h)**
  - Projektinitialisierung & DDL-Skripte: 5 h
  - Backend: Verbindungsaufbau, JDBC, DAO-Pattern: 10 h
  - Frontend: JavaFX FXML, Event Handling: 10 h
  - Algorithmik (Adaptive Wiederholfolge): 7 h
- **4. Qualitätsmanagement (9 h)**
  - Unittesting, Boundary-Checks & Refactoring: 9 h
- **5. Abschluss & Dokumentation (14 h)**
  - Ausformulierung der IHK-Projektdokumentation: 12 h
  - Erstellung Entwicklerkommentare & Abnahme: 2 h

II Listing

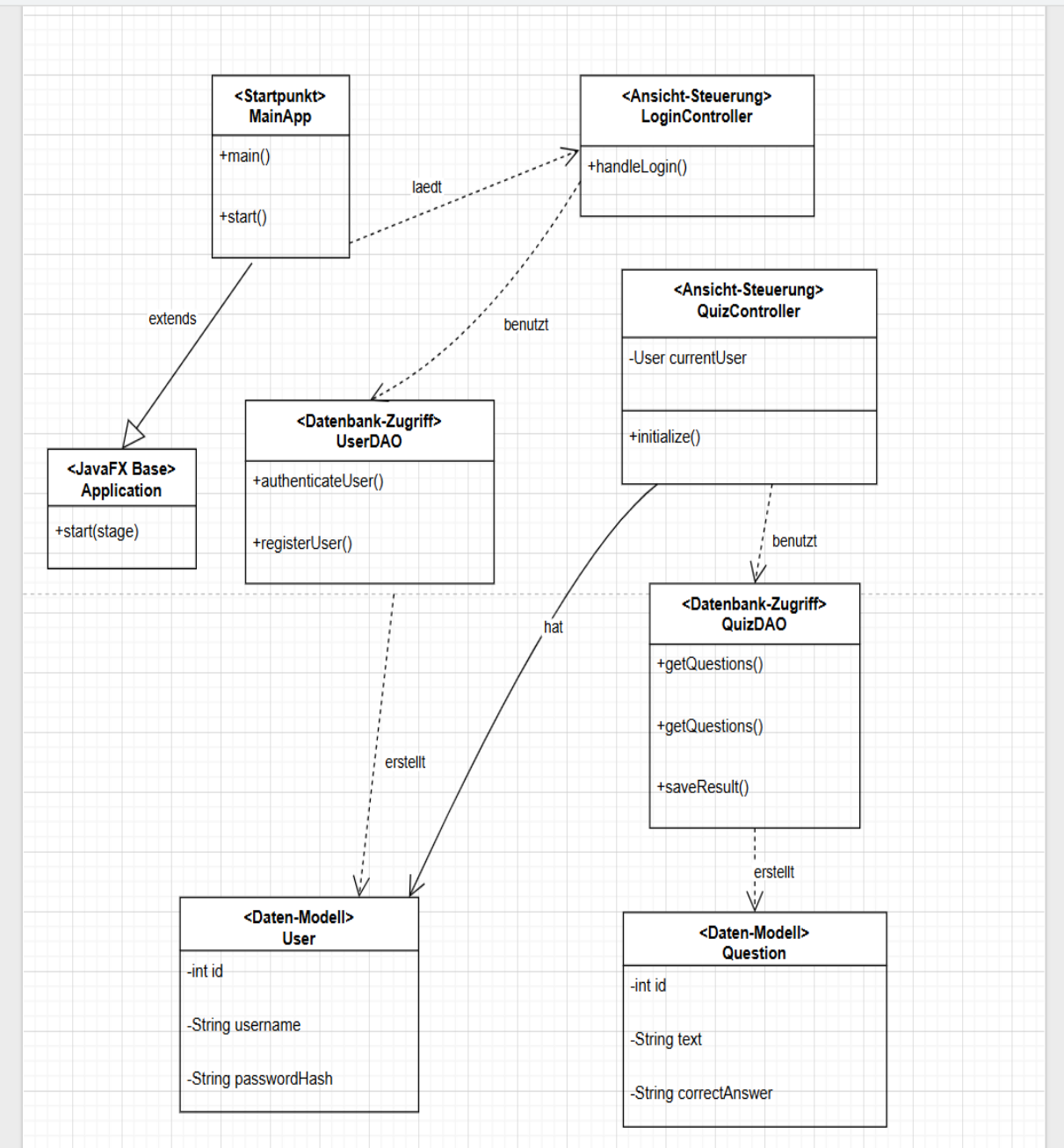
Das grafische Use-Case-Diagramm verdeutlicht die Nutzerinteraktionen im MVC-Rahmen.



Das Entity-Relationship-Diagramm modelliert die relationale Architektur der Tabellen users, questions und user_results.



Dieses Klassendiagramm belegt die strikte Trennung von Oberfläche, Daten-Repräsentation und Controller-Logik.



III Listing

Auszug aus Controller-Logik (Prüfungsbewertung & Leaderboard)

```
private void processAnswer(String selectedOption) {  4 usages  Jalal Sbai *
    disableAnswerButtons(true);
    boolean isCorrect = selectedOption.equalsIgnoreCase(currentQuestion.getCorrectAnswer());

    if (isCorrect) {
        feedbackLabel.setText("Richtig!");
        feedbackLabel.setStyle("-fx-text-fill: green;");
    } else {
        feedbackLabel.setText("Falsch! Richtig wäre: " + currentQuestion.getCorrectAnswer());
        feedbackLabel.setStyle("-fx-text-fill: red;");
    }
    // Vermeidung von Null-Pointern und Speichern in DAO
    if (currentUser != null) {
        quizDAO.saveResult(currentUser.getId(), currentQuestion.getId(), isCorrect);
        refreshLeaderboard();
    }

    btnNext.setVisible(true);
    btnNext.setDisable(false);
}

@FXML  1 usage  Jalal Sbai
public void handleNext(ActionEvent event) {
    currentQuestionIndex++;
    showQuestion(currentQuestionIndex);
}

private void disableAnswerButtons(boolean disable) {  4 usages  Jalal Sbai
    btnOptionA.setDisable(disable);
    btnOptionB.setDisable(disable);
    btnOptionC.setDisable(disable);
    btnOptionD.setDisable(disable);
}
```